Práctica 1 · Servidores, Proxies y Certificados

índice

🌐 NimbusDocs — Portal Documental Seguro Multi-Marca

- 🎯 Objetivo General
- 🧭 Cómo está organizada esta práctica
- I. Diseño y Requisitos
- II. Fase 1 — Backend Apache Multi-Marca (UD2)
- III. Fase 2 — Terminación TLS en Nginx (UD1 + UD3)
- IV. Fase 2 — Seguridad y Límite de Peticiones (UD2 + UD3)
- V. Fase 3 — Diagnóstico Provocado
- VI. Entregables y Observación Directa

🌐 NimbusDocs — Portal Documental Seguro Multi-Marca

🎯 Objetivo General

NimbusDocs es una empresa ficticia que aloja documentación técnica de dos marcas distintas bajo la misma infraestructura física. Vais a diseñar y levantar, desde cero, la arquitectura completa que lo hace posible: primero un Apache que sirve cada marca desde su propio Virtual Host, y más adelante un Nginx delante que termina TLS y añade una capa de seguridad perimetral.

Esto no es el laboratorio de clase repetido con otro nombre. Los dominios, los puertos internos, los nombres de fichero sensibles y los fallos que vais a provocar a propósito en el bloque de diagnóstico los elegís y justificáis vosotros — un dominio o valor copiado literalmente del guion de clase (sitioa.iaw.local, sitiob.iaw.local, app.iaw.local, wp-config.php) se considera una entrega incompleta.

🧭 Cómo está organizada esta práctica

No hace falta —ni se puede— hacerla entera desde el primer día. Está dividida en tres fases que avanzan al mismo ritmo que la clase, así que en cada momento solo necesitáis lo que ya hemos visto:

Fase	Qué incluye
Fase 1 — Apache	Los dos Virtual Hosts, MPM, protección de recursos (bloque II)
Fase 2 — Nginx	Proxy inverso, TLS, cabeceras de seguridad, rate limiting (bloques III y IV)
Fase 3 — Cierre	Diagnóstico provocado sobre el conjunto y defensa oral (bloques V y VI)

✅ Entrega parcial de la Fase 1 (no evaluable, solo para vosotros)
Cuando tengáis la Fase 1 terminada (probablemente esta misma semana), enseñádmela en clase o traedla a tutoría. No cuenta para nota, pero os confirmo si vais bien encaminados antes de que se acumule todo el trabajo al final.

I. Diseño y Requisitos

Antes de tocar una terminal, documentad en el README estas decisiones — la observación directa empezará precisamente por aquí:

- Los dos dominios ficticios elegidos (uno por marca) y por qué los habéis nombrado así.
- El MPM de Apache elegido y la justificación técnica de esa elección (repasad UD2, bloque de MPM).

> ⚠️ **Regla de originalidad**
> Ningún nombre de dominio, puerto, fichero sensible o parámetro de configuración puede coincidir literalmente con los usados en el guion de clase o en las diapositivas. Se audita con un simple grep sobre vuestro repositorio.

II. Fase 1 — Backend Apache Multi-Marca (UD2)

Esto es exactamente lo que acabamos de ver en clase, aplicado a vuestro propio escenario. Un único Apache tiene que servir las dos marcas como dos Virtual Hosts basados en nombre, cada uno con su propio DocumentRoot, ErrorLog y CustomLog:

- Demostrad con curl, usando la cabecera `Host` explícita, que cada dominio devuelve el contenido correcto.
- La marca "interna" (la que decidáis que no debe ser pública) tiene que proteger un fichero sensible simulado — con nombre propio, no `wp-config.php` — y desactivar el listado de directorios en toda su ruta.
- Un dominio que no coincida con ningún Virtual Host debe caer en un comportamiento controlado por vosotros (no el primero al azar) — explicad en el README por qué pasa eso.

> 💡 **Si no sabéis por dónde empezar**
> Es el mismo patrón sites-available/sites-enabled y a2ensite que acabamos de hacer en clase, solo que con dos marcas propias en vez de sitioa/sitiob. Si os atascáis, releed el guion de la sesión de VirtualHost antes de preguntar — la secuencia de pasos es idéntica, solo cambian los nombres.

Con esto termina la Fase 1. Podéis parar aquí hasta que veamos Nginx la semana que viene — no hace falta ni conviene adelantar la Fase 2 sin haberlo visto en clase.

III. Fase 2 — Terminación TLS en Nginx (UD1 + UD3)

A partir de aquí necesitáis lo que veremos la semana que viene. Nginx se coloca delante de vuestro Apache de la Fase 1, recibe todo el tráfico HTTPS y es el único servicio con certificado TLS:

1. Generar vuestro propio certificado de laboratorio (curva ECDSA, igual que en UD1) con el CN de una de vuestras dos marcas.
2. Configurar el bloque server de Nginx con ssl_protocols restringido a TLS 1.2 y 1.3 (nada de versiones obsoletas) y un cipher suite razonado en el README.
3. Redirigir automáticamente todo el tráfico HTTP (puerto 80) a HTTPS con un código de estado correcto — justificad en el README por qué ese código y no otro.
4. Nginx debe reenviar la petición a Apache preservando la cabecera `Host` original — sin eso, Apache no puede resolver a qué marca corresponde la petición. Probadlo primero SIN esa línea para ver el fallo con vuestros propios ojos antes de añadirla.
5. Verificar con openssl s_client el protocolo y cifrado negociados, y adjuntar la salida real (no una captura de pantalla del guion) en el README.

IV. Fase 2 — Seguridad y Límite de Peticiones (UD2 + UD3)

Sobre la misma arquitectura, añadid una capa de seguridad perimetral en Nginx: cabeceras de seguridad y un límite de peticiones razonado a partir de un escenario real, no inventado por vosotros desde cero.

- Cabeceras de seguridad mínimas: `Strict-Transport-Security`, `X-Content-Type-Options` y una tercera que elijáis y justifiquéis vosotros (repasad UD1, tabla de cabeceras).

> 📐 **Escenario de carga a resolver (rate limiting)**
> El equipo de NimbusDocs ha medido que un usuario legítimo leyendo documentación genera como máximo 3 peticiones por segundo de forma sostenida, con picos breves de hasta 6 peticiones/segundo al recargar varias pestañas a la vez. El mes pasado detectaron un script de scraping haciendo más de 20 peticiones por segundo contra la misma ruta. Calculad y justificad en el README los valores de rate y burst de vuestro limit_req_zone que bloqueen ese patrón de scraping sin penalizar a un usuario legítimo en pico.

- Demostrad con una ráfaga de peticiones repetidas (un bucle `for` con curl, o `ab` desde un contenedor cliente) que el límite calculado realmente bloquea, adjuntando la salida.

Con esto termina la Fase 2. La última semana ya es solo integrar, provocar fallos y preparar la defensa.

V. Fase 3 — Diagnóstico Provocado

Este bloque es el que más se observará en la defensa oral. Debéis provocar deliberadamente dos errores de configuración, cada uno de un bloque distinto — no pueden ser variantes del mismo tema:

- 1 fallo relacionado con la parte Apache (Fase 1: Virtual Host, protección de recursos).
- 1 fallo relacionado con la parte Nginx (Fase 2: TLS, comunicación con Apache, o rate limiting).

> 🚫 **Qué NO cuenta como fallo válido**
> Parar el servicio, borrar un fichero entero, o un simple error de tecleo en una ruta no cuentan: tienen que ser fallos que obliguen a entender un concepto de la unidad para diagnosticarlos (p. ej. por qué un Virtual Host no se selecciona aunque el Host coincide, por qué el rate limiting no bloquea aunque la zona está definida).

Para cada uno, documentad en el README una tabla con estas cuatro columnas:

Síntoma observado	Hipótesis inicial	Comando de diagnóstico usado	Causa real y solución
(ejemplo a rellenar por vosotros)	…	…	…

VI. Entregables y Observación Directa

En clase, cada alumno hace una defensa individual breve en la que:

- Arranca su propia infraestructura desde cero (o la tiene ya corriendo) y demuestra en vivo que funciona.
- Explica, sin mirar el README, por qué tomó las decisiones de diseño de la sección I.
- Resuelve en vivo uno de estos dos cambios menores, elegido en el momento por el profesor (preparad los dos, solo se os pedirá uno):
- Añadir un tercer dominio de prueba y demostrar cómo se resuelve (o no) contra vuestros Virtual Hosts actuales.
- Cambiar el valor de rate del limit_req_zone y demostrar con curl que el nuevo límite se aplica.

Como los dos posibles cambios están anunciados aquí, podéis practicarlos de antemano — no es una improvisación a ciegas, es comprobar que entendéis lo que habéis construido.

Preguntas con las que hay que estar preparado para defenderse en la observación directa (ejemplos, no es una lista cerrada):

- ¿Qué pasaría con vuestra configuración si Nginx dejara de reenviar la cabecera Host a Apache?
- ¿Por qué habéis elegido ese MPM y no otro para este escenario concreto?
- ¿Cómo llegasteis al valor de rate y burst del rate limiting a partir del escenario dado?